

Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 01. Patrones de diseño creacionales

Tema:

Introducción

- *Introducción*
- S – Single Responsibility Principle (SRP)
- O – Open/Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

Introducción

Code Smells

- Los code smells son señales observables que pueden revelar problemas más profundos de diseño. No son errores por sí mismos: deben analizarse dentro de su contexto antes de decidir si corresponde una refactorización.
- STUPID agrupa seis problemas frecuentes:
 - S — Singleton: uso abusivo del patrón y del estado global.
 - T — Tight Coupling: acoplamiento fuerte entre componentes.
 - U — Untestability: diseño difícil de probar de forma aislada.
 - P — Premature Optimization: optimización antes de conocer el problema real.
 - I — Indescriptive Naming: nombres que no comunican intención.
 - D — Duplication: conocimiento o lógica repetidos.
- Otros code smells frecuentes
 - Inflación de métodos y clases
 - Obsesión primitiva
 - Demasiados parámetros
 - Feature envy
 - Intimidad inapropiada
 - Cadena de mensajes

Introducción

Introducción a SOLID

- Robert C. Martin consolidó cinco principios orientados al diseño de software orientado a objetos
 - SOLID busca favorecer diseños:
 - Comprensibles y mantenibles.
 - Flexibles ante cambios razonables.
 - Con responsabilidades delimitadas.
 - Con menor acoplamiento entre componentes.
 - Más fáciles de probar y extender.
 - **Advertencia:** SOLID no es un conjunto de reglas absolutas. Debe aplicarse cuando el contexto y el costo del cambio lo justifiquen.
- S | Single Responsibility Principle (SRP) | Responsabilidad única
 - O | Open/Closed Principle (OCP) | Abierto/cerrado
 - L | Liskov Substitution Principle (LSP) | Sustitución de Liskov
 - I | Interface Segregation Principle (ISP) | Segregación de interfaces
 - D | Dependency Inversion Principle (DIP) | Inversión de dependencias

Martin, R. C. (2002). Agile Software Development: Principles, Patterns, and Practices.

Tema:

S – Single Responsibility Principle (SRP)

- Introducción
- *S – Single Responsibility Principle (SRP)*
- O – Open/Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

S – Single Responsibility Principle (SRP)

Una razón para cambiar

- “Una clase debe tener una sola razón para cambiar.”
- Un módulo debe responder ante un solo actor o grupo de interesados.
- En el ejemplo, Persona concentra decisiones que pueden cambiar por motivos diferentes:
 - Reglas del dominio: cálculo de la edad.
 - Persistencia: almacenamiento en archivos.
 - Presentación: forma de mostrar la información.
- Consecuencia: un cambio técnico en el almacenamiento puede afectar innecesariamente a una clase del dominio.
- SRP no significa: una clase con un solo método, pocas líneas o una única tarea trivial.

Persona
- codigo : int
- nombre : String
- fechaNacimiento : LocalDate
+ calcularEdad():int
+ almacenarEnArchivo(): void
+ mostrarInformacion(): void

S – Single Responsibility Principle (SRP)

Separar por razones de cambio

- Aplicar SRP implica identificar decisiones que cambian por actores o motivos diferentes.
 - Persona conserva los datos y reglas propias del dominio.
 - PersonaArchivo se ocupa de guardar y recuperar personas.
- Cada componente puede evolucionar por una razón diferente:
 - Cambios en las reglas del negocio.
 - Cambios en el mecanismo de persistencia.
 - Cambios en la interfaz o formato de salida.
- Resultado: mayor cohesión y menor impacto de los cambios.



S – Single Responsibility Principle (SRP)

Actividad en grupo: detectar responsabilidades

- Una tienda en línea posee una clase PedidoService que:
 - Valida los productos y cantidades.
 - Calcula descuentos e impuestos.
 - Guarda el pedido en la base de datos.
 - Procesa el pago.
 - Envía la confirmación al cliente.
 - Genera un archivo PDF con la factura.
- Actividad del grupo
 - Identifiquen los actores o motivos que pueden provocar cambios.
 - Determinen qué responsabilidades deberían permanecer juntas.
 - Propongan una reorganización mediante un diagrama UML.
 - Justifiquen cada separación utilizando SRP.
 - Señalen una separación que sería innecesaria o excesiva.
- Uso permitido de IA
 - Utilicen la IA como revisora de la propuesta. El grupo debe registrar una recomendación que haya aceptado, modificado o rechazado y explicar su decisión.
- Entrega: diagrama UML y explicación oral de tres decisiones de diseño.

Tema:

O – Open/Closed Principle (OCP)

- Introducción
- S – Single Responsibility Principle (SRP)
- *O – Open/Closed Principle (OCP)*
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

O – Open/Closed Principle (OCP)

Principio: Abierto para extensión, cerrado para modificación.

- Las entidades de software deberían permitir incorporar nuevos comportamientos sin modificar continuamente su código estable.
- Abierto para extensión: admite nuevas variantes de comportamiento.
- Cerrado para modificación: incorporar nuevas variantes no debería obligar a modificar repetidamente el núcleo estable.
- Normalmente se logra mediante abstracciones, composición y polimorfismo.
- No significa que una clase nunca pueda modificarse.
- Conviene aplicarlo en puntos donde existe una variación probable y relevante.

Problema

- Un servicio envía notificaciones por correo o SMS utilizando condicionales. Cada nuevo canal obliga a modificar la misma clase, agregar otra condición y volver a probar todo el flujo.

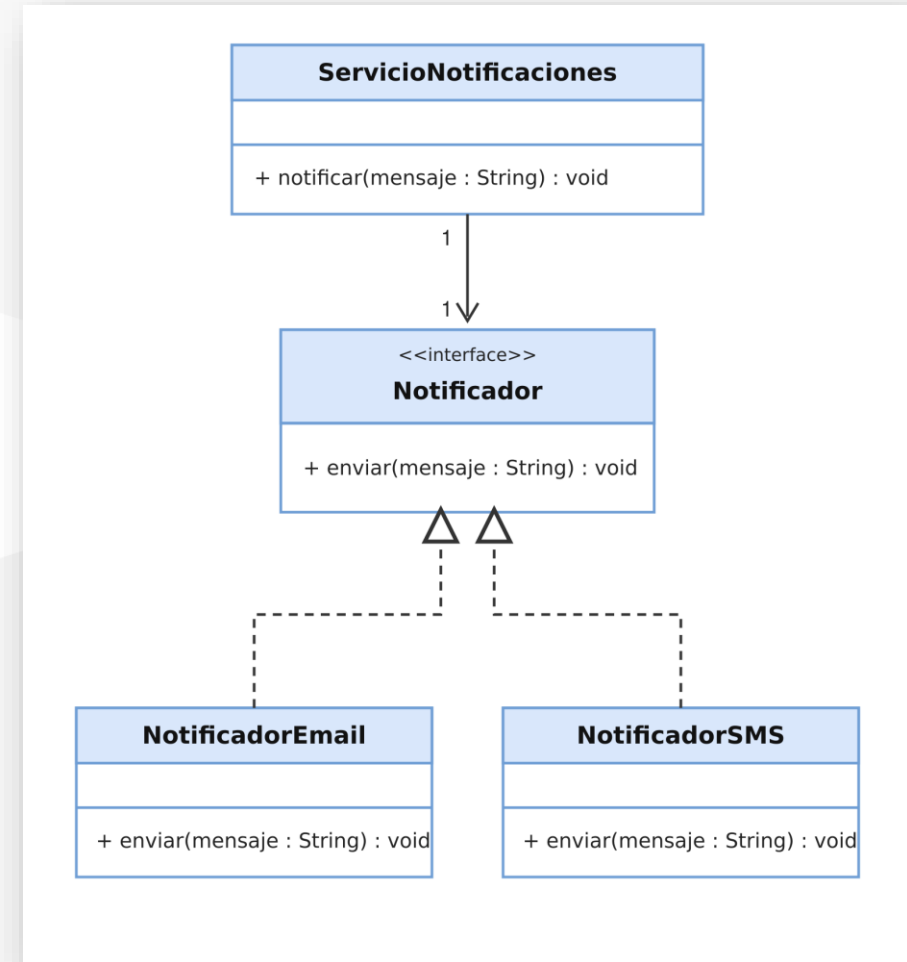
Señal de alerta

- La misma clase cambia repetidamente por la aparición de nuevas variantes.

O – Open/Closed Principle (OCP)

Conclusiones

- Ante nuevas variantes de comportamiento, buscamos extender el diseño sin modificar repetidamente el núcleo estable.
- MODIFICAR puede implicar:
 - Añadir nuevos atributos.
 - Añadir nuevos métodos.
 - Cambiar el comportamiento de métodos.
- EXTENDER puede implicar:
 - Crear una abstracción.
 - Agregar una subclase.
 - Crear una clase de relación intermedia.
 - Utilizar patrones de diseño.
- OCP debe aplicarse donde exista variación probable y relevante.
- Corregir un defecto o cambiar una regla del negocio puede requerir modificar código existente.



O – Open/Closed Principle (OCP)

Actividad en grupo: extender sin modificar

- Un sistema calcula el costo de entrega mediante un if:
 - Producto físico: costo según peso y distancia.
 - Producto digital: costo cero.
- El negocio incorporará próximamente:
 - Entrega refrigerada.
 - Entrega internacional.
 - Retiro en tienda.
- Actividad del grupo
 - Identifiquen qué comportamiento varía.
 - Determinen qué parte debería permanecer estable.
 - Propongan una abstracción.
 - Representen la solución mediante UML.
 - Expliquen cómo agregarían una nueva modalidad sin modificar el cálculo existente.
- ¿Se justifica aplicar OCP ahora o todavía sería una abstracción prematura?

Tema:

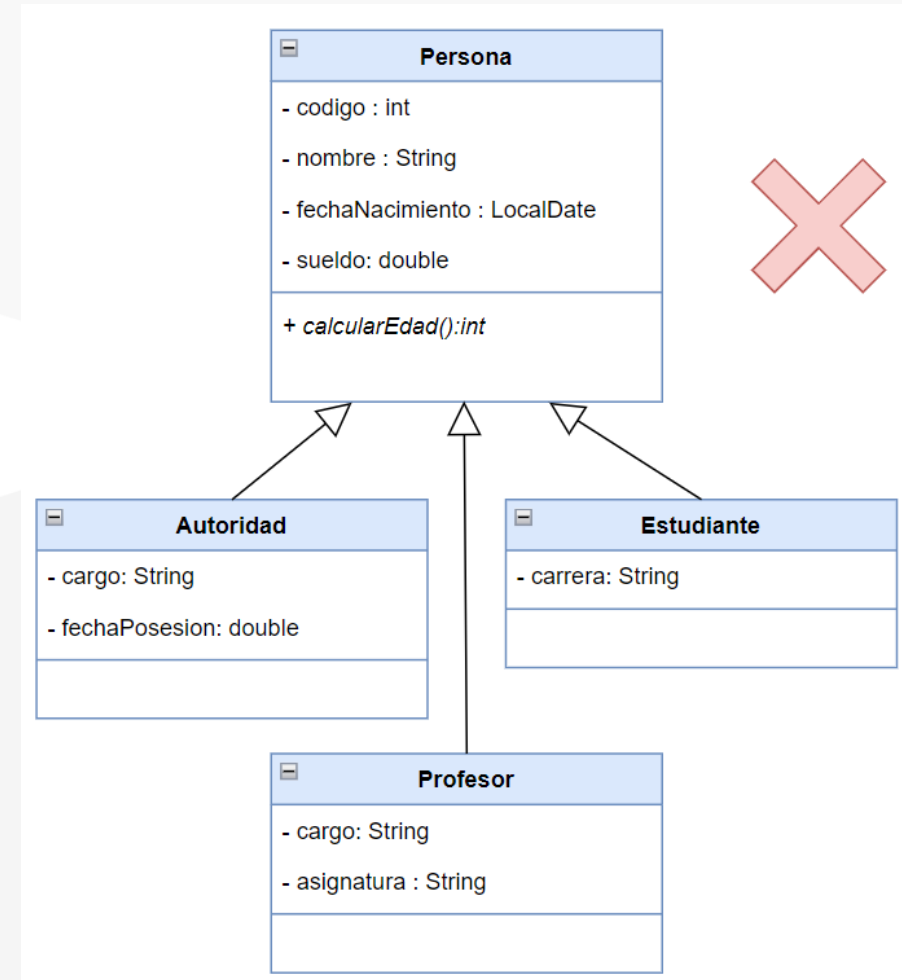
L – Liskov Substitution Principle (LSP)

- Introducción
- S – Single Responsibility Principle (SRP)
- O – Open/Closed Principle (OCP)
- *L – Liskov Substitution Principle (LSP)*
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

L – Liskov Substitution Principle (LSP)

Principio

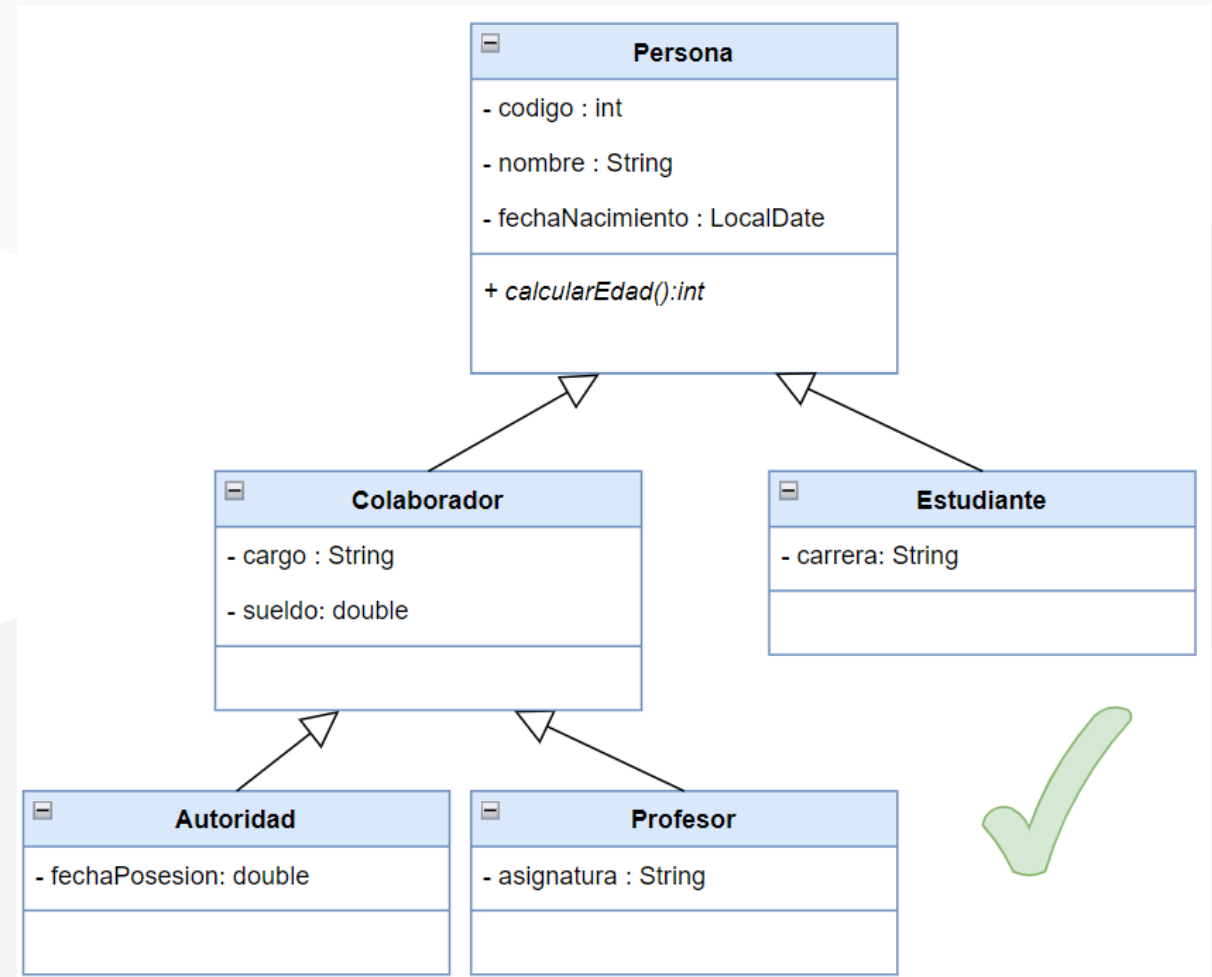
- Los objetos de un subtipo deben poder sustituir a los objetos del tipo base sin alterar el comportamiento esperado del sistema.
 - El subtipo debe respetar el contrato definido por la abstracción.
 - No debe exigir precondiciones más restrictivas.
 - Debe conservar los invariantes relevantes.
 - No debería introducir excepciones inesperadas.
 - LSP no exige comportamientos idénticos; exige que todos respeten las expectativas del contrato.
- Señal de alerta
 - El código cliente necesita preguntar por el tipo concreto, evitar determinados subtipos o capturar errores que la abstracción no contempla.



L – Liskov Substitution Principle (LSP)

LSP en Java

- En Java, además del contrato comportamental de LSP, la sobrescritura está sujeta a reglas específicas del lenguaje.
- Los tipos y el orden de los parámetros deben coincidir con los definidos en el método de la clase base.
- El tipo de retorno debe ser igual o compatible mediante un tipo covariante.
- Una sobrescritura no puede reducir la visibilidad del método heredado.
- Las excepciones verificadas no pueden ser más generales que las declaradas por el método base.
- La anotación `@Override` permite verificar que realmente se está sobrescribiendo un método.



Tema:

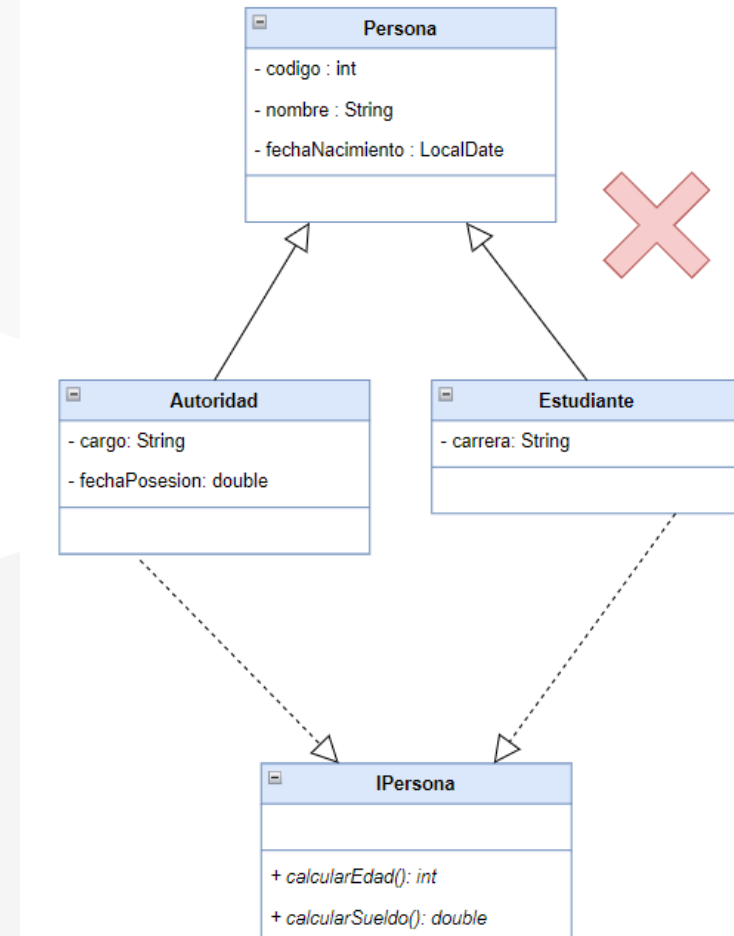
I – Interface Segregation Principle (ISP)

- Introducción
- S – Single Responsibility Principle (SRP)
- O – Open/Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- **I – Interface Segregation Principle (ISP)**
- D – Dependency Inversion Principle (DIP)

I – Interface Segregation Principle (ISP)

Principio

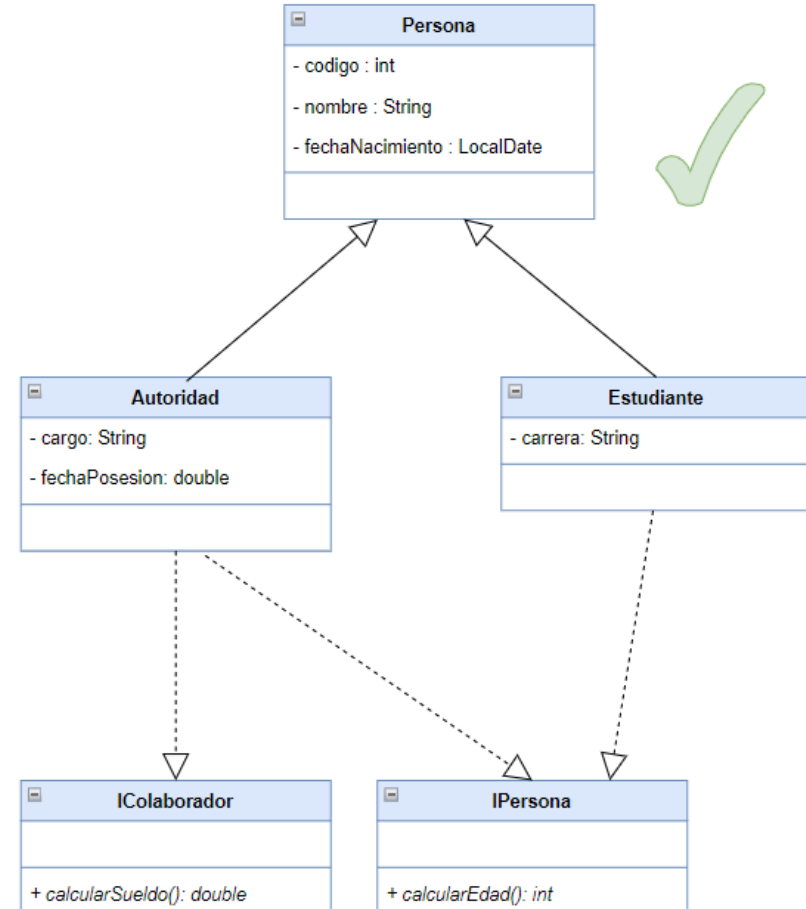
- I – Interface Segregation Principle (ISP)
 - “Make fine-grained interfaces that are client specific.”
 - Los clientes no deberían estar obligados a depender de interfaces que no utilicen.
 - Una interfaz debe contener únicamente las operaciones que resultan coherentes para los clientes que dependen de ella.
- Señales de alerta
 - Clases obligadas a implementar métodos que no necesitan.
 - Métodos implementados vacíos o que lanzan excepciones porque no aplican.
 - Cambios en una interfaz que afectan a clientes que no utilizan las operaciones modificadas.
 - Interfaces con demasiadas responsabilidades.



I – Interface Segregation Principle (ISP)

Conclusiones

- Dividir las interfaces demasiado amplias en interfaces más pequeñas, coherentes y específicas para sus clientes.
- Cada cliente debería depender únicamente de las operaciones que realmente necesita.
- Las clases implementan contratos acordes con sus responsabilidades, evitando métodos que no tienen sentido para ellas.
- Un cambio en una interfaz no debería afectar innecesariamente a clientes que no utilizan la funcionalidad modificada.
- ISP no implica crear una interfaz por cada método; la separación debe responder a necesidades reales de los clientes..



Tema:

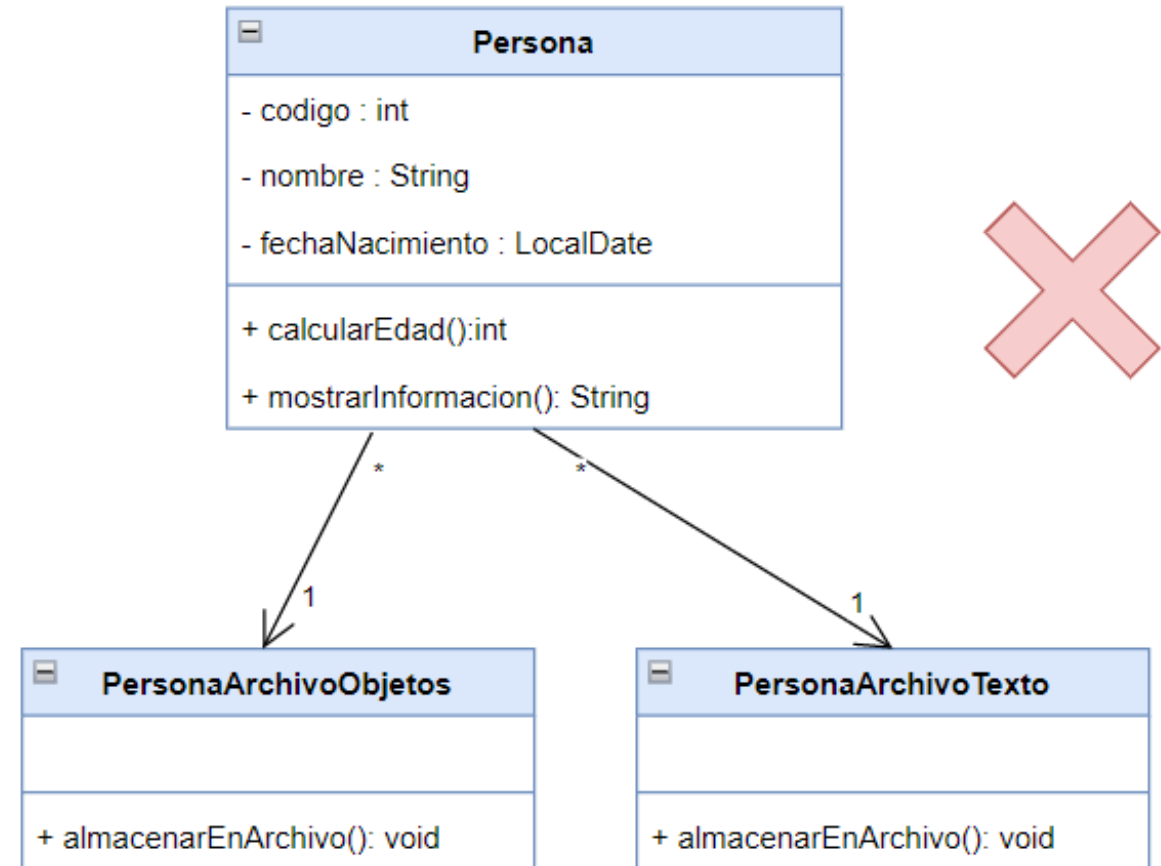
D – Dependency Inversion Principle (DIP)

- Introducción
- S – Single Responsibility Principle (SRP)
- O – Open/Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- *D – Dependency Inversion Principle (DIP)*

D – Dependency Inversion Principle (DIP)

Principio

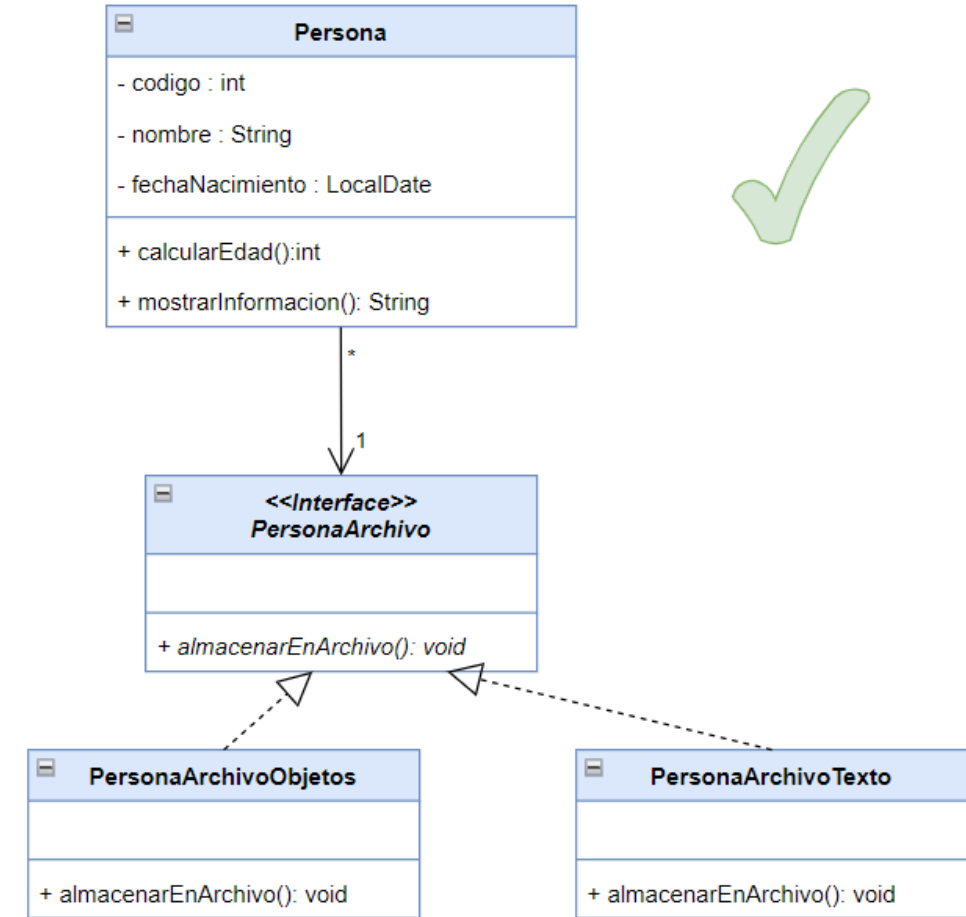
- “Depend on abstractions, not on concretions.”
- Depend de abstracciones significa depender de contratos estables, no de implementaciones concretas.
- En Java, estos contratos suelen representarse mediante interfaces o clases abstractas.
- Los módulos de alto nivel (lógica de negocio) no deberían depender directamente de módulos de bajo nivel (persistencia, red, GUI, servicios externos, etc.).
- Las abstracciones no deberían depender de los detalles.
- Los detalles deberían depender de las abstracciones.
- Esto reduce el acoplamiento con tecnologías concretas y facilita sustituir implementaciones.



D – Dependency Inversion Principle (DIP)

Conclusiones

- Los módulos de alto nivel contienen las reglas y decisiones principales del negocio.
- Los módulos de bajo nivel proporcionan detalles tecnológicos, como interfaz de usuario, persistencia, red o integración con servicios externos.
- Las reglas del negocio no deberían depender directamente de esos detalles tecnológicos.
- Los detalles deben adaptarse a los contratos definidos por las necesidades del sistema.
- Esto permite sustituir tecnologías con menor impacto sobre la lógica principal.



Unidad 01.05

Retroalimentación y cierre

Qué vimos hoy

- Los code smells son señales que requieren análisis, no pruebas automáticas de mal diseño.
- SRP organiza responsabilidades según sus razones de cambio.
- OCP permite incorporar variantes sin modificar repetidamente un núcleo estable.
- LSP exige que los subtipos respeten el contrato de la abstracción.
- ISP evita que los clientes dependan de operaciones innecesarias.
- DIP orienta las dependencias hacia abstracciones estables.
- SOLID debe aplicarse según el contexto, el cambio esperado y el costo de la abstracción.
- Aplicar principios mecánicamente puede generar fragmentación y sobreingeniería.

Antes de finalizar: ¿Qué principio resultó más difícil de distinguir y qué decisión del diseño grupal modificarían ahora?

Idea final: Un buen diseño no utiliza la mayor cantidad posible de abstracciones; utiliza las necesarias para controlar los cambios relevantes sin complicar innecesariamente el sistema.

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Anexos

Links de Referencia

- <https://codideep.com/blogpost/stupid-vs-solid>
- <https://refactoring.guru/design-patterns>
- <https://reactiveprogramming.io/books/design-patterns/es>
- https://es.wikipedia.org/wiki/Barbara_Liskov
- [https://hmong.es/wiki/Coupling_\(computer_science\)](https://hmong.es/wiki/Coupling_(computer_science))

Gracias por su atención

posgrados.ups.edu.ec